

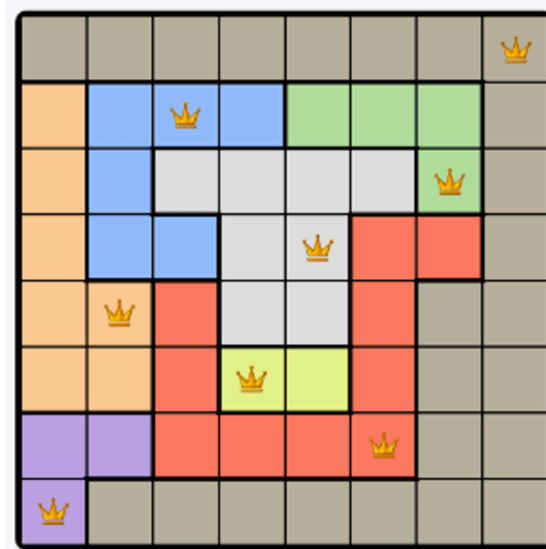
Laporan Tugas Kecil 1
IF2211 Strategi Algoritma
Penyelesaian Permainan *Queens* LinkedIn



Disusun oleh:
Nama : Keisha Rizka Syofyani
NIM : 13524073
Prodi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2025

1. Penjelasan Algoritma

Pada tugas kecil 1 Strategi Algoritma ini kita diminta untuk membuat program yang dapat menemukan satu solusi pada Queens, gim logika yang tersedia pada situs jejaring profesional LinkedIn, dengan menggunakan algoritma *brute force*. Tujuan dari game ini adalah menempatkan queen pada sebuah papan persegi berwarna sehingga terdapat hanya satu queen pada tiap baris, kolom, dan daerah warna. Selain itu, satu queen tidak dapat ditempatkan bersebelahan dengan queen lainnya, termasuk secara diagonal.



Berikut tahapan yang dilakukan pada program pencarian solusi pada game ini:

1.1 Input dan Inisiasi Informasi Papan

Informasi papan warna di-load dari file berformat teks (.txt) yang disimpan di dalam folder test dengan memastikan validasi penamaan file dan ketersediaan file di dalam folder. Informasi ini kemudian dikonversi menjadi sebuah matriks 2 dimensi bernama **papan_warna** dan program menyimpan set dari warna-warna unik papan pada sebuah array **set_warna** sebagai panduan untuk mendapatkan jumlah *queen* yang akan ditempatkan pada papan nantinya. Program ini diimplementasikan di dalam sebuah fungsi **load_file()** yang akan return **papan_warna** dan **set_warna** tersebut.

1.2 Pencarian Semua Kemungkinan Solusi dengan Permutasi

Pencarian semua kemungkinan solusi ini diimplementasikan dengan menggunakan fungsi **permutasi()**. Fungsi ini digunakan untuk mencari semua kemungkinan penempatan N *queen* pada papan temp yang ukurannya sama dengan **papan_warna** yang sudah diinisialisasi pada baris dan kolom yang berbeda. Fungsi ini menggunakan parameter **jumlah_queen** dan **jumlah_kolom** serta akan menyimpan hasil operasinya di dalam array **semua_kemungkinan**. Fungsi ini menggunakan fungsi lokal **generate()**

berupa fungsi rekursif untuk mengiterasi penempatan queen pada papan hingga semua queen berhasil ditempatkan.

1.3 Evaluasi Solusi yang Sesuai Aturan

Program akan mengevaluasi semua kemungkinan solusi yang didapatkan pada tahap sebelumnya untuk mendapatkan sebuah solusi yang tepat sesuai dengan syarat yang valid, yakni queen ditempatkan pada baris, kolom, warna yang berbeda serta tidak bertetangga bersebelahan maupun secara diagonal. Tahapan ini digunakan beberapa fungsi pembantu, yakni:

- Fungsi **cek_posisi_aman()** : fungsi yang akan mengecek apakah posisi pada baris dan kolom tertentu di sebuah **papan_warna** aman untuk ditempatkan sebuah queen dengan syarat tidak berada pada baris, kolom, warna, tetangga bersebelahan, serta diagonal yang sama. Fungsi ini memiliki parameter **baris**, **kolom**, **papan_temp**, **papan_warna** dan akan return nilai Boolean (**True** jika *queen* aman untuk diposisikan dan **False** jika tidak aman dan syarat tidak terpenuhi).
- Fungsi **cek_kemungkinan_aman()** : fungsi yang akan mengecek apakah salah satu kemungkinan solusi yang dihasilkan pada hasil permutasi tadi valid dengan mengevaluasinya menggunakan fungsi **cek_posisi_aman()** pada setiap iterasi posisi di papan. Fungsi ini menggunakan parameter **kemungkinan** (salah satu papan kemungkinan solusi), **jumlah_baris**, **jumlah_kolom**, dan **papan_warna** yang akan return Boolean (**True** jika papan valid sesuai syarat dan **False** jika tidak valid).
- Fungsi **temukan_kemungkinan_aman()** : fungsi yang akan mengambil 1 solusi pertama valid yang ditemukan dari **semua_kemungkinan** solusi dari hasil permutasi sesuai evaluasi syarat yang dilakukan oleh fungsi **cek_kemungkinan_aman()**. Fungsi ini menggunakan parameter **semua_kemungkinan**, **papan_warna**, **jumlah_baris**, dan **jumlah_kolom** serta akan return sebuah papan yang valid jika ditemukan atau **None** jika tidak ditemukan satupun solusi valid hingga akhir iterasi.

1.4 Hitung Jumlah Kasus yang Dievaluasi dan Runtime

Untuk tracking progress dari pencarian solusi dari game ini, dibuatkan juga Live Update dari proses dengan menggunakan status progress setiap 5% beserta tampilan papan yang sedang dicek. Setiap iterasi pencarian solusi juga dihitung jumlah kasus yang dievaluasi dengan penjumlahan setiap iterasi serta perhitungan

waktu yang dibutuhkan selama proses dengan menggunakan library time. Program ini diletakkan pada fungsi `temukan_kemungkinan_aman()`.

1.5 Menampilkan Hasil Pencarian Solusi Akhir

Setelah menemukan pencarian solusi, program dapat menampilkan hasil solusi papan berupa CLI papan maupun gambar. Program dapat menampilkan ilustrasi gambar papan menggunakan karakter pembentuk dan diberikan styling pewarnaan menggunakan library colorama. Selain itu, program dapat membuat ilustrasi gambar papan seperti pada game *Queens* dengan format .png yang dapat ditampilkan menggunakan library matplotlib. Untuk sistem pewarnaan papan digunakan pewarnaan random menggunakan library random, sedangkan untuk notasi Queen dibuat berbeda dan terkhusus, yakni background berwarna putih dan foreground kuning.

1.6 Menyimpan Hasil Solusi

Di bagian akhir program, dimungkinkan untuk dapat menyimpan hasil pencarian solusi papan disimpan ke dalam sebuah file .txt serta gambar papan dalam format .png sesuai nama file yang diinput user. File ini akan disimpan di dalam folder default program, yakni di dalam folder test.

2. Source Code Implementasi Program

Program yang dibuat dipecah menjadi beberapa fungsi dan prosedur bantuan dan juga sebuah program utama

2.1 Library yang Digunakan

```
import os
import time
import random
import matplotlib.pyplot as plt
import colorama
from colorama import Fore, Back, Style
```

2.2 Fungsi dan Prosedur Terkait

```
# ===== Fungsi-Fungsi Terkait
=====

# Fungsi yang mencari semua kemungkinan solusi yang bisa dihasilkan
dari permutasi N buah queen ke dalam papan pada baris dan kolom yang
berbeda
```

```

def permutasi(jumlah_queen, jumlah_kolom):
    # Simpan semua kemungkinan hasil di dalam sebuah array
    semua_kemungkinan = []
    kolom_dipakai = [False] * jumlah_kolom
    posisi = [-1] * jumlah_queen

    # Fungsi Rekursif Pembantu untuk menempatkan Queen di baris
    def generate(baris):
        # Basis : semua queen sudah diposisikan
        if baris == jumlah_queen:

            papan_temp = [ "." for _ in range(jumlah_kolom) ]
            for r in range(jumlah_baris):
                for r in range(jumlah_queen):
                    papan_temp[r][posisi[r]] = "#"

            semua_kemungkinan.append(papan_temp)
            return

        # Rekursif
        for kolom in range(jumlah_kolom):
            if not kolom_dipakai[kolom]:
                kolom_dipakai[kolom] = True
                posisi[baris] = kolom
                generate(baris + 1)
                kolom_dipakai[kolom] = False

    generate(0)
    return semua_kemungkinan

# Fungsi Boolean untuk mengecek posisi tertentu pada papan aman
# untuk menempatkan queen
def cek_posisi_aman(baris, kolom, papan_temp, papan_warna):
    current = papan_warna[baris][kolom]

    # cek apakah ada di warna / baris / kolom yang sama
    for i in range (jumlah_baris):
        for j in range (jumlah_kolom):
            if papan_temp[i][j] == "#":
                # Skip posisi saat ini

```

```

        if i == baris and j == kolom:
            continue

        # Cek warna
        if papan_warna[i][j] == current:
            return False

        # Cek baris
        if i == baris :
            return False

        # Cek kolom
        if j == kolom:
            return False

    # cek apakah bersebelahan
    for jarak_baris in [-1, 0, 1]:
        for jarak_kolom in [-1, 0, 1]:
            # Lewati cuurent kotak
            if jarak_baris == 0 and jarak_kolom == 0:
                continue

            baris_tetangga, kolom_tetangga = baris + jarak_baris,
kolom + jarak_kolom
            if 0 <= baris_tetangga < jumlah_baris and 0 <=
kolom_tetangga < jumlah_kolom:
                if papan_temp[baris_tetangga][kolom_tetangga] ==
"#":
                    return False

    # Selain kondisi syarat:
    return True

# Fungsi Boolean untuk mengecek apakah sebuah papan valid sesuai
syarat permainan
def cek_kemungkinan_aman(kemungkinan, jumlah_baris, jumlah_kolom,
papan_warna):
    for baris in range(jumlah_baris):
        for kolom in range(jumlah_kolom):
            if kemungkinan[baris][kolom] == "#":

```

```

        if not cek_posisi_aman(baris, kolom, kemungkinan,
papan_warna):

            return False

    return True

# Fungsi yang mengambil solusi valid pertama dari semua kemungkinan
solusi
def temukan_kemungkinan_aman(semua_kemungkinan, papan_warna,
jumlah_baris, jumlah_kolom):
    total = len(semua_kemungkinan)

    for idx, kemungkinan in enumerate(semua_kemungkinan):

        # Live Update
        if idx % max(1, total // 20) == 0: # update tiap 5%
            persen = (idx / total) * 100
            print(Fore.YELLOW + f"[Progress] {persen:.1f}%
({idx}/{total}) kemungkinan dicek")
            show_papan(kemungkinan, jumlah_baris, jumlah_kolom)

        # Cari kemungkinan hasil yang aman
        if cek_kemungkinan_aman(kemungkinan, jumlah_baris,
jumlah_kolom, papan_warna):
            print(Fore.GREEN + f"\nSolusi ditemukan pada kemungkinan
ke-{idx}")

            return kemungkinan

    # Jika tidak ditemukan solusi
    print(Fore.RED + "Tidak ditemukan kemungkinan solusi")
    return None

# Fungsi Helper
# Load File txt papan
def load_file():
    folder = "../test/"
    # Input nama file untuk papan
    file_name = input("Masukkan nama file (format .txt) : ")
    if not file_name.endswith(".txt"):

```

```

        file_name += ".txt"

    full_path = os.path.join(folder, file_name)

    while not os.path.exists(full_path):
        print(Fore.YELLOW + f"File '{file_name}' tidak ditemukan di
folder test ini.")
        file_name = input("Masukkan nama file kembali: ")
        full_path = os.path.join(folder, file_name)

    with open(full_path, 'r') as file:
        # Mengambil matriks papan warna dari file
        papan_warna = [list(line.strip()) for line in file]

        # Menyimpan set warna yang ada
        set_warna = set()
        for baris in papan_warna:
            for elemen in baris:
                set_warna.add(elemen)

    return papan_warna, set_warna

def save_file(papan_final, kasus, durasi, jumlah_queen):
    folder = "../test/"
    file_name = input("Masukkan nama file yang akan disimpan : ")

    # Validasi File txt
    # Tambahkan .txt otomatis jika user lupa mengetiknya
    if not file_name.endswith(".txt"):
        file_name += ".txt"

    full_path = os.path.join(folder, file_name)

    # Validasi Jika File memiliki nama serupa
    while os.path.exists(full_path):
        print(Fore.YELLOW + f"Peringatan: File '{file_name}' sudah
ada!")

        pilihan = input(Fore.YELLOW + "Apakah ingin menimpa
(overwrite) file tersebut? (Ya/Tidak): ").capitalize()

```

```

        if pilihan == "Ya":
            break # Keluar dari loop while dan lanjut simpan
(overwrite)
        else:
            # Jika tidak ingin menimpa, minta nama file baru
            file_name = input("Masukkan nama file baru: ")
            if not file_name.endswith(".txt"):
                file_name += ".txt"
            full_path = os.path.join(folder, file_name)

# Validasi File Gambar
# Ambil tanpa ".txt"
base_name = os.path.splitext(full_path)[0]
image_file = base_name + ".png"
path_image = os.path.join(folder, image_file)

# Simpan Gambar
plt.savefig(image_file, bbox_inches="tight", dpi=300)

with open(full_path, 'w') as f_out:

f_out.write("=====\n")
    f_out.write("===== Solusi Games Queens LinkedIn
=====\\n")

f_out.write("=====\n")

    f_out.write(f"{jumlah_queen} Queens berhasil
diposisikan\\n\\n")
    f_out.write("Hasil Penempatan Queens:\\n")
    for baris in papan_final:
        # Menggabungkan list menjadi string lalu menambah enter
(\\n)

        baris_string = "".join(baris) + "\\n"
        f_out.write(baris_string)

    f_out.write(f"\\nDurasi pencarian : {durasi*100:.2f} ms\\n")

```

```

        f_out.write(f"Banyak kasus yang ditinjau : {kasus} kasus")

    show_nama_file = os.path.splitext(file_name)[0]
    print(f"\nHasil penempatan Queen pada papan berhasil disimpan
pada file {show_nama_file}.txt dan visualisasi pada
{show_nama_file}.png\n")

# Tampilan CLI Menarik
# Menampilkan Papan di CLI
def show_papan(papan, jumlah_baris, jumlah_kolom):
    # Garis pembatas horizontal
    garis_horizontal = "  " + "-" * jumlah_kolom

    # Cetak nomor kolom
    print("    ", end="")
    for c in range(jumlah_kolom):
        print(f" {c} ", end="")
    print()

    # Mapping warna random
    pilihan_warna = [
        Back.RED,
        Back.GREEN,
        Back.YELLOW,
        Back.BLUE,
        Back.MAGENTA,
        Back.CYAN,
    ]

    warna_map = {}
    for baris in papan:
        for cell in baris:
            if cell != "#" and cell not in warna_map:
                warna_map[cell] = random.choice(pilihan_warna)

    for i, baris_data in enumerate(papan):
        print(Style.BRIGHT + garis_horizontal)
        # Cetak nomor baris di samping
        print(f"{i} |", end="")
        for cell in baris_data:

```

```

        # Jika cell adalah Queen, beri spasi ekstra agar
        menonjol

        if cell == "#":
            print(Back.WHITE + Fore.YELLOW + Style.BRIGHT + " #
", Fore.RESET + Style.BRIGHT + "|", sep="", end="")
        else:
            style = warna_map[cell]
            print(style + warna_map[cell] + f" {cell} " +
Back.RESET + Style.BRIGHT + "|", end="")
        print()
        print(garis_horizontal)

# Membuat gambar dari papan
def papan_to_image(papan, papan_warna):
    n_baris = len(papan)
    n_kolom = len(papan[0])

    # map warna huruf -> warna random
    unik = sorted(set(c for row in papan_warna for c in row))
    cmap = {u: (random.random(), random.random(), random.random()) for
u in unik}

    fig, ax = plt.subplots()

    for i in range(n_baris):
        for j in range(n_kolom):
            rect = plt.Rectangle(
                (j, n_baris-i-1),
                1,1,
                facecolor=cmap[papan_warna[i][j]],
                edgecolor="black"
            )
            ax.add_patch(rect)

            if papan[i][j] == "#":
                ax.text(j+0.5, n_baris-i-0.5, "♔",
                    ha="center", va="center", fontsize=18)

    ax.set_xlim(0, n_kolom)
    ax.set_ylim(0, n_baris)

```

```

    ax.set_aspect("equal")
    ax.axis("off")
    plt.show()

# Closing Statement Program saar Program Berakhir
def closing_statement():
    print(Fore.YELLOW +
"=====
==")
    print(Fore.YELLOW + "===== " + Fore.RESET + "👑 Terima
kasih telah bermain Queens LinkedIn 👑" + Fore.YELLOW + "
=====")
    print(Fore.YELLOW +
"=====
==")

```

2.3 Program Utama

```

# ===== Program Utama
=====

# Opening Statement Program
print(Fore.YELLOW +
"=====
")
print(Fore.YELLOW + "===== " + Fore.RESET + "👑 Selamat Datang
di Queens LinkedIn 👑" + Fore.YELLOW + " =====")
print(Fore.YELLOW +
"=====
")

# 1. Input nama file papan
papan_warna, set_warna = load_file()

# 2. Tampilkan papan yang akan disolve
jumlah_baris = len(papan_warna)
jumlah_kolom = len(papan_warna[0])

print(Fore.YELLOW + "\n===== Papan Warna Queens =====")
show_papan(papan_warna, jumlah_baris, jumlah_kolom)

```

```

print("Jumlah baris : ", jumlah_baris)
print("Jumlah kolom : ", jumlah_kolom)

# 3. Jalankan Program Utama : Pencarian dan Pemosisian Queen
opsi_lanjutan = input("Lanjutkan proses penempatan Queen? (Ya/Tidak) : ")

while opsi_lanjutan != "Ya" and opsi_lanjutan != "Tidak":
    print(Fore.YELLOW + "Pilihan tidak valid, ulangi masukkan Anda!")
    opsi_lanjutan = input("Lanjutkan proses penempatan Queen? (Ya/Tidak) : ")

if opsi_lanjutan == "Tidak":
    closing_statement()

else:
    print("\nMencari posisi queen yang sesuai...")

    # Mulai waktu pencarian
    mulai = time.time()

    # Proses Iterasi Pencarian
    jumlah_queen = len(set_warna) # Jumlah queen = jumlah anggota set warna
    semua_kemungkinan = permutasi(jumlah_queen, jumlah_kolom)

    # Hitung jumlah kasus yang dicek
    kasus = len(semua_kemungkinan)

    hasil = temukan_kemungkinan_aman(semua_kemungkinan, papan_warna, jumlah_baris, jumlah_kolom)

    # catat durasi pencarian
    selesai = time.time()
    durasi = selesai - mulai

    if hasil == None:
        print(Fore.RED + "Papan Queens tidak dapat diselesaikan")

    else:

```

```

print(Fore.GREEN + "Queen sudah berhasil diposisikan")

# Salin hasil penempatan queen ke papan warna
papan_final = []
for i in range(jumlah_baris):
    baris_papan = []
    for j in range(jumlah_kolom):
        if hasil[i][j] == "#":
            baris_papan.append("#")
        else:
            baris_papan.append(papan_warna[i][j])
    papan_final.append(baris_papan)

# Hasil akhir
print(Fore.YELLOW +
"\n=====")
print(Fore.YELLOW + "==== Hasil Akhir Penempatan Queen pada
Papan Warna =====")
print(Fore.YELLOW +
"=====")

# Menampilkan Hasil Penempatan Queens
show_papan(papan_final, jumlah_baris, jumlah_kolom)

# Menampilkan Gambar Papan Queens
papan_to_image(papan_final, papan_warna)

print(f"Durasi pencarian : {durasi*100:.2f} ms")
print(f"Banyak kasus yang ditinjau : {kasus} kasus")

# Menyimpan Hasil Solusi
opsi_simpan = input("Apakah Anda ingin menyimpan solusi?
(Ya/Tidak) : ")
while opsi_simpan != "Ya" and opsi_simpan != "Tidak":
    print(Fore.YELLOW + "Pilihan tidak valid, ulangi
masukkan Anda!")
    opsi_simpan = input("Apakah Anda ingin menyimpan solusi?
(Ya/Tidak) : ")

if opsi_simpan == "Ya":

```

```

        save_file(papan_final, kasus, durasi, jumlah_queen)
    else:
        print("Papan tidak disimpan")

closing_statement()

```

3. Test Case Program

3.1 Inisialisasi Program

```

===== 👑 Selamat Datang di Queens LinkedIn 👑 =====
=====
Masukkan nama file (format .txt) : test

===== Papan Warna Queens =====
  0  1  2  3  4  5  6  7  8
+---+---+---+---+---+---+---+---+
0 | A | A | B | B | C | C | C | C | D |
+---+---+---+---+---+---+---+---+
1 | A | B | B | B | B | C | E | C | D |
+---+---+---+---+---+---+---+---+
2 | A | B | B | B | D | C | E | C | D |
+---+---+---+---+---+---+---+---+
3 | A | A | A | B | D | C | C | C | D |
+---+---+---+---+---+---+---+---+
4 | B | B | B | B | D | D | D | D | D |
+---+---+---+---+---+---+---+---+
5 | F | G | G | G | D | D | H | D | D |
+---+---+---+---+---+---+---+---+
6 | F | G | I | G | D | D | H | D | D |
+---+---+---+---+---+---+---+---+
7 | F | G | I | G | D | D | H | D | D |
+---+---+---+---+---+---+---+---+
8 | F | G | G | G | D | D | H | H | H |
+---+---+---+---+---+---+---+---+
Jumlah baris : 9
Jumlah kolom : 9
Lanjutkan proses penempatan Queen? (Ya/Tidak) : 

```

3.2 Pencarian Semua Kemungkinan Solusi

Mencari posisi queen yang sesuai...

[Progress] 0.0% (0/362880) kemungkinan dicek

	0	1	2	3	4	5	6	7	8
0	#	.	.	.	.	.	.	.	.
1	.	#	.	.	.	.	.	.	.
2	.	.	#	.	.	.	.	.	.
3	.	.	.	#	.	.	.	.	.
4	.	.	.	.	#	.	.	.	.
5	.	.	.	.	.	#	.	.	.
6	.	.	.	.	.	.	#	.	.
7	.	.	.	.	.	.	.	#	.
8	.	.	.	.	.	.	.	.	#

[Progress] 5.0% (18144/362880) kemungkinan dicek

	0	1	2	3	4	5	6	7	8
0	#	.	.	.	.	.	.	.	.
1	.	.	.	.	#	.	.	.	.
2	.	.	.	.	.	.	#	.	.
3	.	.	#	.	.	.	.	.	.
4	.	.	.	#	.	.	.	.	.
5	.	#	.	.	.	.	.	.	.
6	.	.	.	.	.	#	.	.	.

[Progress] 80.0% (290304/362880) kemungkinan dicek

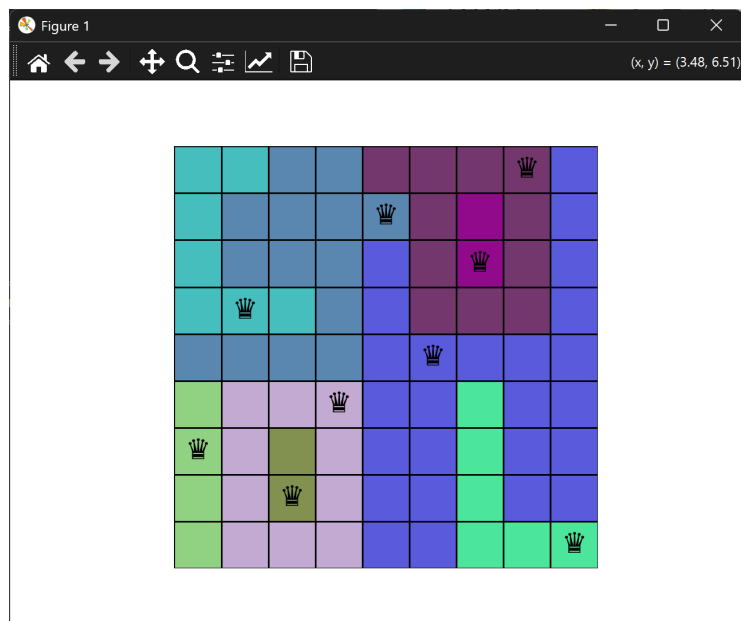
	0	1	2	3	4	5	6	7	8
0	.	.	.	.	.	.	.	#	.
1	.	#	.	.	.	.	.	.	.
2	.	.	.	.	.	#	.	.	.
3	.	.	#	.	.	.	.	.	.
4	.	.	.	#	.	.	.	.	.
5	#	.	.	.	.	.	.	.	.
6	.	.	.	.	#	.	.	.	.
7	.	.	.	.	.	.	#	.	.
8	.	.	.	.	.	.	.	.	#

Solusi ditemukan pada kemungkinan ke-306204
Queen sudah berhasil diposisikan

3.3 Menampilkan Solusi Valid

```
=====
==== Hasil Akhir Penempatan Queen pada Papan Warna ====
=====
   0  1  2  3  4  5  6  7  8
+---+---+---+---+---+---+---+---+
0 | A | A | B | B | C | C | C | # | D |
+---+---+---+---+---+---+---+---+
1 | A | B | B | B | # | C | E | C | D |
+---+---+---+---+---+---+---+---+
2 | A | B | B | B | D | C | # | C | D |
+---+---+---+---+---+---+---+---+
3 | A | # | A | B | D | C | C | C | D |
+---+---+---+---+---+---+---+---+
4 | B | B | B | B | D | # | D | D | D |
+---+---+---+---+---+---+---+---+
5 | F | G | G | # | D | D | H | D | D |
+---+---+---+---+---+---+---+---+
6 | # | G | I | G | D | D | H | D | D |
+---+---+---+---+---+---+---+---+
7 | F | G | # | G | D | D | H | D | D |
+---+---+---+---+---+---+---+---+
8 | F | G | G | G | D | D | H | H | # |
+---+---+---+---+---+---+---+---+
Durasi pencarian : 531.07 ms
Banyak kasus yang ditinjau : 362880 kasus
```

3.4 Menampilkan Solusi dalam Bentuk Gambar



3.5 Simpan Solusi dan Terminasi Program

```
=====
Apakah Anda ingin menyimpan solusi? (Ya/Tidak) : Ya
Masukkan nama file yang akan disimpan : solusi_test

Hasil penempatan Queen pada papan berhasil disimpan pada file :
solusi_test.txt dan visualisasi pada solusi_test.png

=====
===== 🏰 Terima kasih telah bermain Queens LinkedIn 🏰 =====
=====
```

4. Lampiran

3.1 Repository Program

Source code program dapat diakses pada repository Github berikut ini:

https://github.com/keirzka/Tucil1_13524073.git

3.2 Checklist Isi Program

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil di jalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Program dapat menyimpan solusi dalam bentuk file gambar	✓	

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

A handwritten signature in black ink, appearing to be 'KR' or similar, on a light-colored background.

Keisha Rizka Syofyani
13524073